signal and image proccecing lab

DR.nouri

electrical engineering department

Ahmadreza Majlesara 400101861
Parsa Hatami 400100962

Lab 6

November 24, 2024

Brain Source Localization

In this experiment, we aim to fully solve the brain source localization problem:

- The model is a three-layer sphere with radii of 8.5, 9.2, and 9.85 centimeters.
- The electrode locations are stored in the file `ElecPosXYZ.mat`. This file contains the coordinates of the electrodes and their numbering order.
- To calculate the exact location of each electrode, multiply its radius by the cosine and sine of the azimuthal and polar angles.
- The location of dipoles is assumed to be volumetric and within the inner sphere (layer one). The resolution for arbitrary dipoles (distance between two adjacent dipoles) can also be specified.
- For easy implementation, the function `ForwardModel_3shell.m` is provided. It accepts input resolution for dipoles and calculates model parameters and outputs for dipoles and the lead-field matrix (transfer matrix).
- The function `ForwardModel_3shell.m` and its usage are explained in the file `main.m`.
- For visualizing the EEG potential distribution, the file `Display_Potential_3D.m` is provided. This function plots the potential on the scalp for the input dipole position.

part 1

Considering a resolution of 1 cm and using the function `ForwardModel_3shell.m`, generate all possible dipole positions and represent them in a 3D space. You can use the function `scatter3` for visualization. Compute the lead-field matrix and save it.

solution

for this part i used the following code:

```
load('ElecPosXYZ');
load('Interictal');

ModelParams.R = [8 8.5 9.2];
layers = [8 8.5 9.2];
ModelParams.Sigma = [3.3e-3 8.25e-5 3.3e-3];
ModelParams.Lambda = [.5979 .2037 .0237];
ModelParams.Mu = [.6342 .9364 1.0362];
Resolution = 1;
[SourceLocations, LeadFieldMatrix] = ForwardModel_3shell(
    Resolution, ModelParams);
scatter3(SourceLocations(1,:), SourceLocations(2,:),
    SourceLocations(3,:));
```

The code generates and visualizes all possible dipole positions in a 3D space, considering a 1 cm resolution. It loads the electrode positions and EEG data, sets up model parameters like the radii of the brain layers and their conductivity properties, and then uses the `ForwardModel_3shell.m` function to calculate dipole locations and the corresponding lead-field matrix.

the output is shown in below figure:

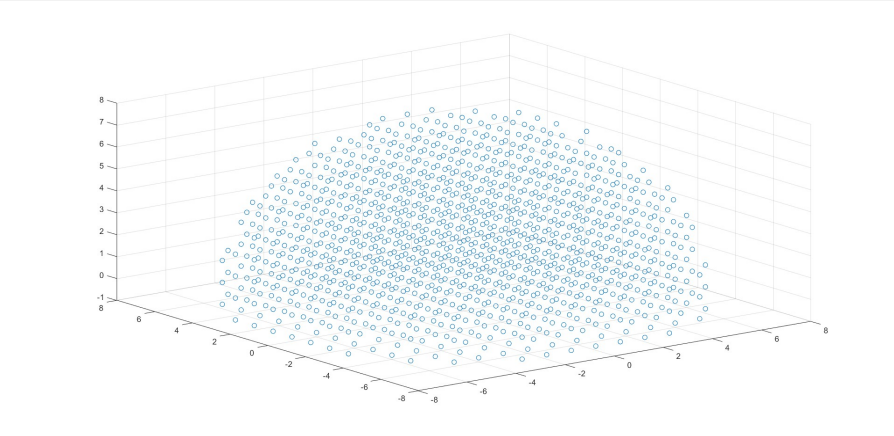

Figure 1. all possible dipole positions

part 2

In the plot from Step A, also display the positions of the electrodes and label each electrode alongside it.

solution

the code for this part is as below:

```
electrode_x = cellfun(@(elec) elec.XYZ(1) * layers(3), ElecPos);
electrode_y = cellfun(@(elec) elec.XYZ(2) * layers(3), ElecPos);
electrode_z = cellfun(@(elec) elec.XYZ(3) * layers(3), ElecPos);
electrode_labels = cellfun(@(elec) elec.Name, ElecPos, '
    UniformOutput', false);

scatter3(SourceLocations(1,:), SourceLocations(2,:),
    SourceLocations(3,:));
hold on
scatter3(electrode_x, electrode_y, electrode_z, 'filled');
text(electrode_x, electrode_y, electrode_z, electrode_labels);
```

The code calculates the electrode positions by scaling their coordinates according to the outermost layer radius and retrieves their names. It visualizes both dipole and electrode locations in a 3D plot using `scatter3`, overlays the electrodes with filled markers, and labels each electrode with its name using the `text` function.
the output of this code can be seen in below figure:

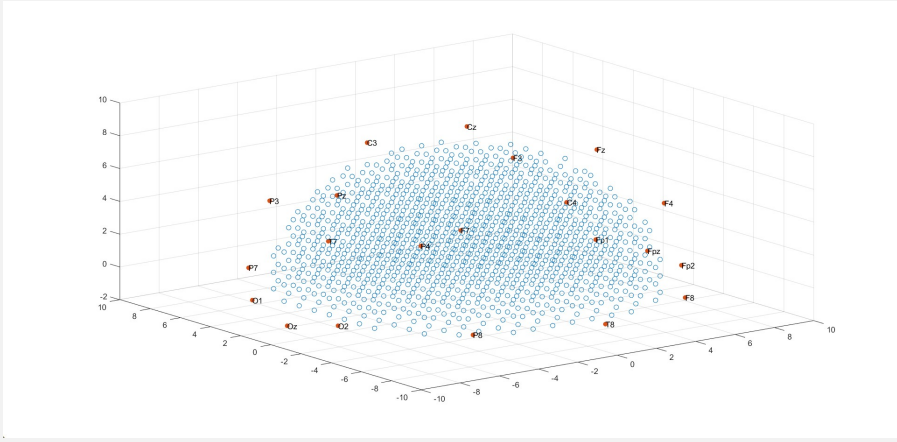

Figure 2. all possible dipole positions with electrodes and their labels

part 3

Choose a random dipole position and specify its azimuthal and polar angles. Mark this dipole position in the plot. Specify the dipole location graphically.

solution

the code for this part is as below:

```
rand_index = int32(1316 * rand(1,1) + 1);

scatter3(SourceLocations(1, rand_index), SourceLocations(2,
    rand_index), SourceLocations(3, rand_index), 'g', 'filled');

line_x = SourceLocations(1, rand_index) + (0:0.01:1);
line_y = SourceLocations(2, rand_index) + (0:0.01:1);
line_z = SourceLocations(3, rand_index) + (0:0.01:1);

scatter3(SourceLocations(1,:), SourceLocations(2,:),
    SourceLocations(3,:));
hold on
scatter3(electrode_x, electrode_y, electrode_z, 'filled');
text(electrode_x, electrode_y, electrode_z, electrode_labels);
hold on
plot3(line_x, line_y, line_z, 'LineWidth', 2);
```

The code selects a random dipole location from `SourceLocations` and calculates a line extending from this location by incrementing its coordinates. The dipole positions and electrodes are visualized, with the electrodes labeled accordingly. The line is then plotted to connect the selected dipole location.
the output of this code is shown in below figure:

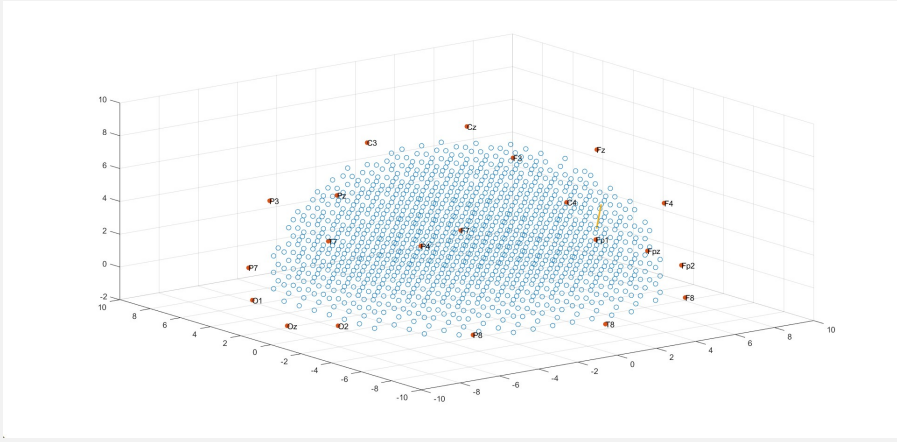

Figure 3. all possible dipole positions with electrodes and their labels and random source location

part 4

Using the file `Interictal.mat`, match the EEG activity with a single row of the matrix that corresponds to a specific dipole position. Specify the direction and magnitude of the dipole and calculate the potential for 21 electrodes. Represent these potentials on the scalp and label each electrode with its corresponding potential in the plot.

solution

for this part i used the code below:

```
interictal_signal = Interictal(5,:);
dipole_direction = [SourceLocations(1,rand_index), SourceLocations
    (2,rand_index), SourceLocations(3,rand_index)]./norm([
    SourceLocations(1,rand_index), SourceLocations(2,rand_index),
    SourceLocations(3,rand_index)]);
M = LeadFieldMatrix(:, (3 * rand_index - 2:3 * rand_index)) *
    dipole_direction' * interictal_signal;

disp_eeg(M, max(abs(M(:))), [], electrode_labels, 'Electrode
    Potentials');
xlim("tight");
```

The code extracts an interictal signal and computes the dipole direction by normalizing the selected dipole's coordinates. It then calculates the electrode potentials by multiplying the relevant rows of the lead-field matrix with the dipole direction and the interictal signal.
the output of this code is shown in below figure:

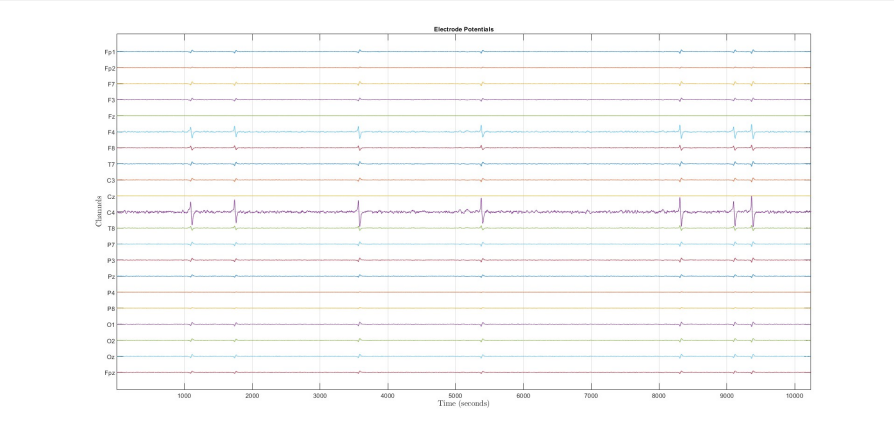

Figure 4. EEG channels of `Interictal.mat`

part 5

When calculating the potential summit for all electrodes (on the scalp), identify the top vertex and create a 7x7 square window centered on the vertex. Compute the average potential for all electrodes within the window and plot the potential distribution around this summit.Save the computed potentials and use the `Display_Potential_3D.m` function to visualize the location of each electrode with a color-coded map.

solution

for this section i used this code:

```
peak_times_all = cell(1, 21);
avg_potential = zeros(1, 21);

for i = 1:21
    [peak_values, peak_times] = findpeaks(M(i,:), 'MinPeakHeight',
        0.3 * max(M(i,:)));
    peak_times_all{i} = peak_times;
end

for i = 1:21
    peak_times = peak_times_all{i};
    window = peak_times + (-3:1:3)';
    valid_window = window(window > 0 & window <= size(M, 2));
    avg_potential(i) = mean(M(i, valid_window), 'all');
end

Display_Potential_3D(ModelParams.R(3), avg_potential);
```

The code identifies the peak times of the electrode potentials exceeding 30% of the maximum value and calculates a 7x7 temporal window around each peak.

It computes the average potential across all valid windows for each electrode. These average potentials are then used to represent the potential distribution on the scalp.
the output is shown in below figure:

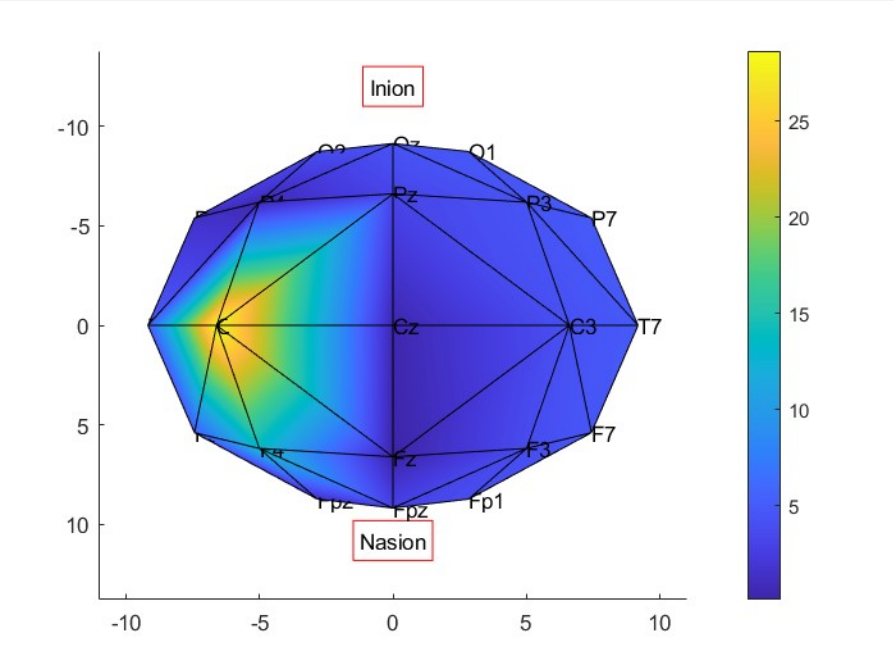

Figure 5. location of each electrode with color-coded map

The image represents the spatial distribution of electrode potentials on the scalp, where the color scale indicates the intensity of the potentials. The highest potential is concentrated in a specific region, shown in yellow, while lower potentials are represented in blue. The labeled electrodes (e.g., Cz, Pz, Fp1) are positioned according to the 3D head model, and the map visually highlights the potential variation across the scalp. Key anatomical landmarks, such as the Nasion and Inion, are also marked for reference.

part 6

Apply the `MNE` algorithm to the computed potentials and solve the inverse problem.

solution

the code i used for this part is as follows:

```
DipoleLocationEstimate = LeadFieldMatrix' * ((LeadFieldMatrix *
    LeadFieldMatrix' + 0.6 * eye(21)) \ M);
```

The `MNE` (Minimum Norm Estimation) algorithm solves the inverse problem by estimating the source dipole location and amplitude that best explain the observed electrode potentials, while minimizing the solution norm. The formula for the algorithm is:

$$\hat{\mathbf{q}} = \mathbf{G}^\top(\mathbf{G}\mathbf{G}^\top + \lambda\mathbf{I})^{-1}\mathbf{M}$$

where:

- $\hat{\mathbf{q}}$: Estimated dipole location and amplitude.
- $\mathbf{G}$: Lead-field matrix.
- $\mathbf{M}$: Observed potentials at the electrodes.
- $\lambda\mathbf{I}$: Regularization term to account for ill-posedness ($\lambda = 0.6$ in the provided code).

The provided code directly implements this formula to compute `DipoleLocationEstimate`, representing the estimated dipole parameters based on the given potentials.

part 7

Determine the location of the source dipole, meaning for each dipole position, calculate the estimation domain range and select the dipole with the largest domain range. Also, obtain the orientation of the dipole.

solution

the code i used for this section is as below:

```
DipoleLocationAvg = max(DipoleLocationEstimate, [], 2);
DipoleLocationAvg_reshaped = reshape(DipoleLocationAvg, 3, []);
DipoleLocationNorm = vecnorm(DipoleLocationAvg_reshaped, 2, 1);

[NormMax, LocationIndex] = max(DipoleLocationNorm);
DipoleDirection = DipoleLocationAvg_reshaped(:, LocationIndex) /
    NormMax;
```

part 8

Using the results of part 6, obtain the error in determining the dipole location and orientation.

solution

for this part i used the code below:

```
position_error = norm(SourceLocations(:, rand_index) -
    SourceLocations(:, LocationIndex));
direction_error = norm(dipole_direction - DipoleDirection);
disp("position error is:" + position_error)
disp("direction error is:" + direction_error)
```

the output of this code is as:

position error is:2.2361
direction error is:2.2074

part 9

Change the dipole location and orientation in part (2) (parts (a) to (c)) and repeat the results to compare the theoretical and practical results (in cases of minimal error in position and orientation).

solution

for this part i used the code below. the difference of 3 parts is in the index of them.

```
[max_value, index_1] = max(vecnorm(SourceLocations, 2, 1));

interictal_signal = Interictal(5,:);
dipole_direction = SourceLocations(:, index_1) / norm(
    SourceLocations(:, index_1));

M = LeadFieldMatrix(:, (3 * index_1 - 2):(3 * index_1)) * (
    dipole_direction .* interictal_signal);

disp_eeg(M, max(abs(M(:))), [], electrode_labels, 'Electrode
    Potentials');
xlim("tight");

avg_potential = zeros(1, 21);
for i = 1:21
    [peak_values, peak_times] = findpeaks(M(i,:), 'MinPeakHeight',
        0.3 * max(M(i,:)));
    window_indices = peak_times + (-3:3)';
    window_indices = window_indices(window_indices > 0 &
        window_indices <= size(M, 2));
    avg_potential(i) = mean(M(i, window_indices), 'all');
end

figure;
Display_Potential_3D(ModelParams.R(3), avg_potential);

DipoleLocationEstimate = LeadFieldMatrix' * ((LeadFieldMatrix *
    LeadFieldMatrix' + 0.6 * eye(21)) \ M);

DipoleLocationAvg = max(DipoleLocationEstimate, [], 2);
DipoleLocationAvg_reshaped = reshape(DipoleLocationAvg, 3, []);
DipoleLocationNorm = vecnorm(DipoleLocationAvg_reshaped, 2, 1);
```

```
[NormMax, LocationIndex] = max(DipoleLocationNorm);
DipoleDirection = DipoleLocationAvg_reshaped(:, LocationIndex) /
    NormMax;

position_error_1 = norm(SourceLocations(:, index_1) -
    SourceLocations(:, LocationIndex));
direction_error_1 = abs(norm(dipole_direction) - norm(
    DipoleDirection));
```

the outputs are shown in the figures 6 to 11

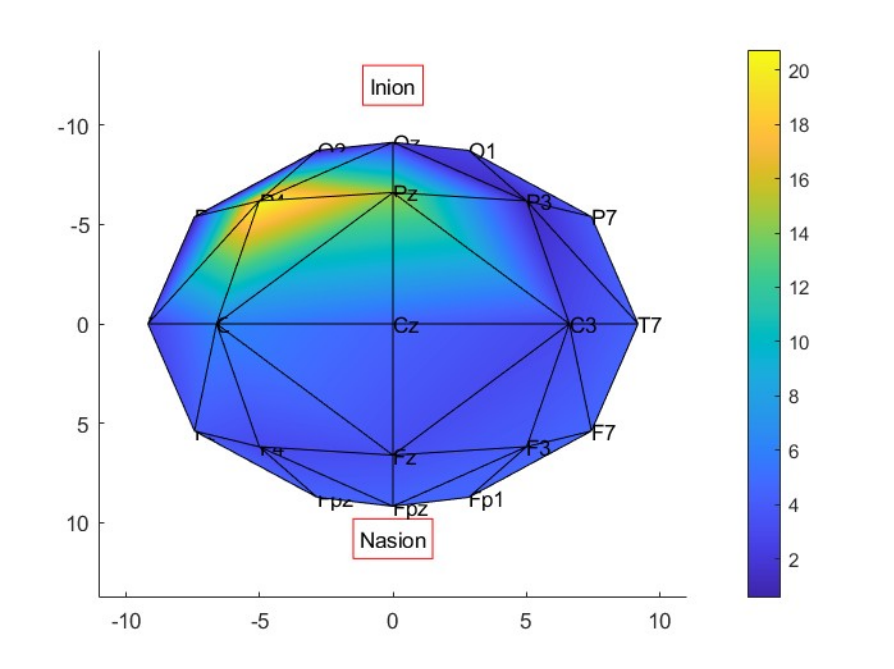

Figure 6. location of each electrode with color-coded map for the cortex

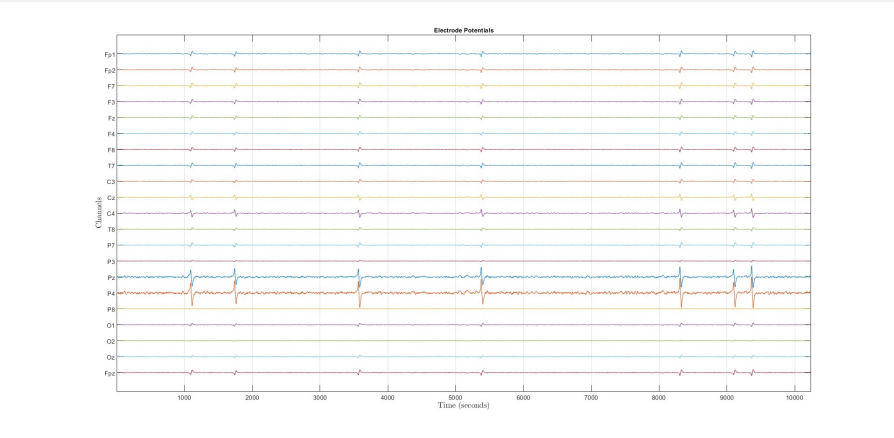

Figure 7. EEG signals corresponding to cortex

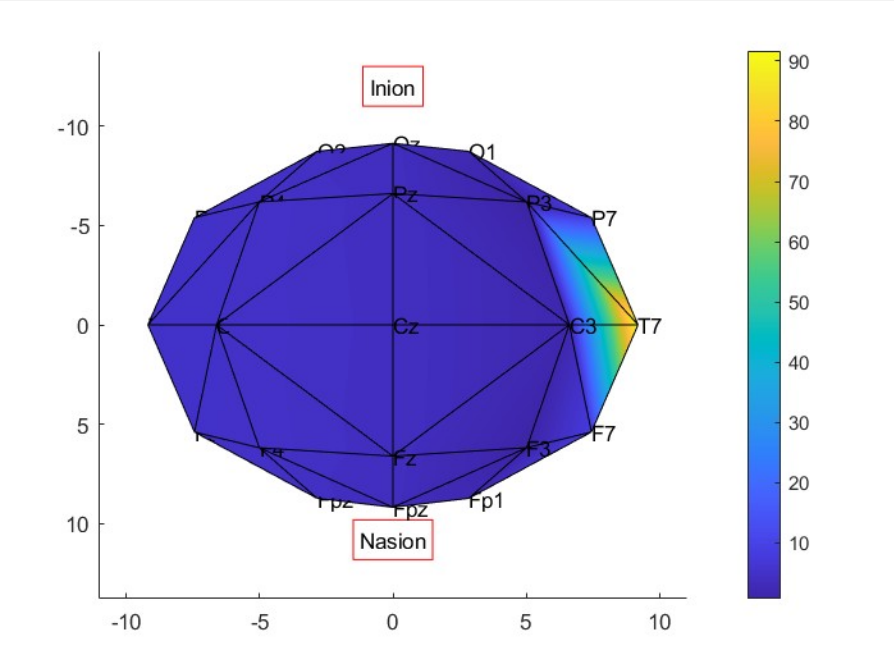

Figure 8. location of each electrode with color-coded map for the temporal

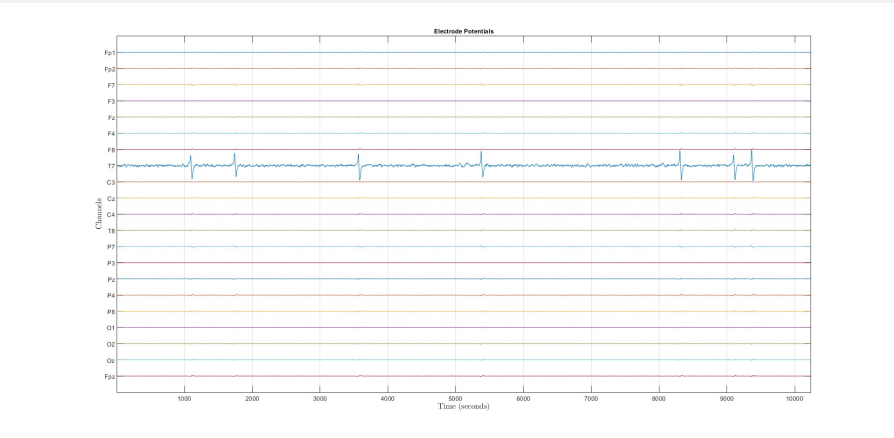

Figure 9. EEG signals corresponding to temporal

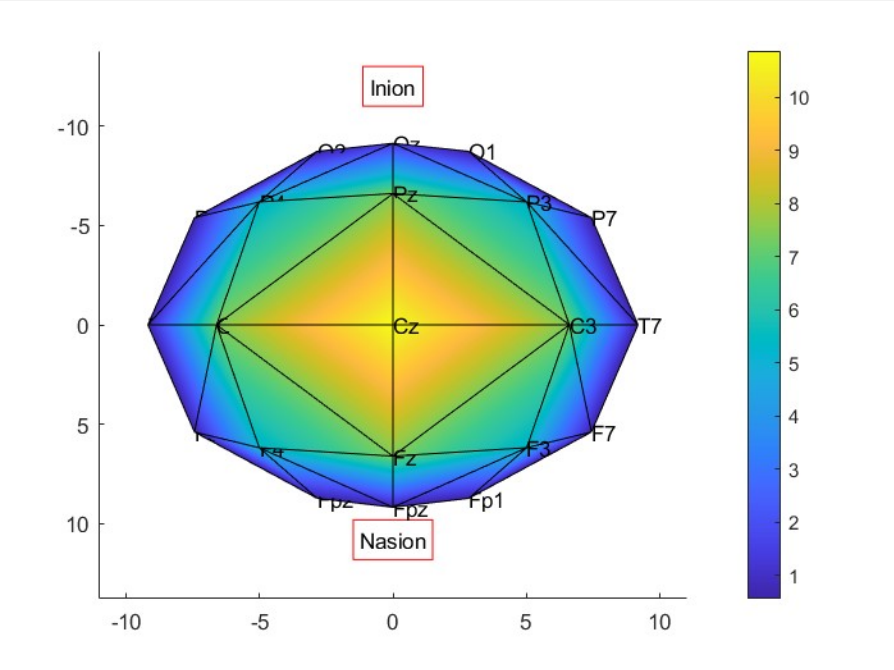

Figure 10. location of each electrode with color-coded map for the depth

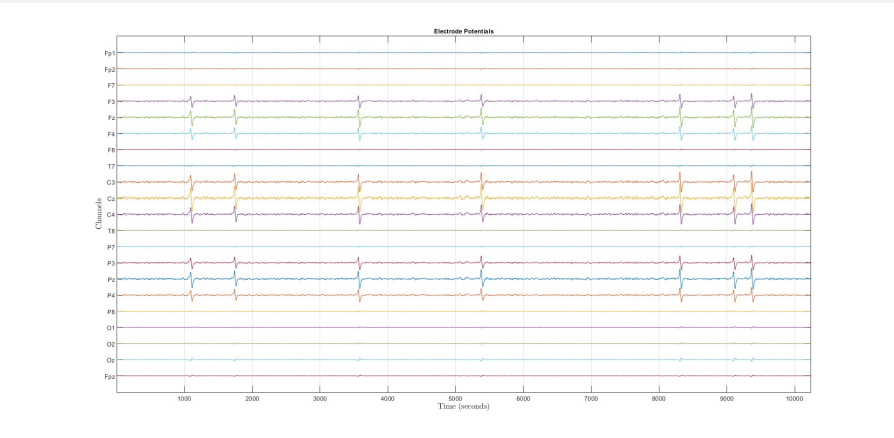

Figure 11. EEG signals corresponding to depth

the errors are as follows:

- cortex:
  - **direction error : 0**
  - **position error : 2.4495**
- temporal:
  - **direction error : 0**
  - **position error : 1**
- depth:
  - **direction error : 0**
  - **position error : 7.8102**

additional

In this test, we aim to solve the inverse problem using the model parameters and considering a dipole as the unknown source. To find the unknown, consider a dipole position as an unknown source (specified by the numbers x, y, and z in the axis space) and three unknowns for the dipole orientation. Using the simulated annealing algorithm, search for the dipole position and orientation in a way that minimizes the cost function $||m - Gq||^2$. Repeat this for parts (d) and (e).

solution

for this part i used this code:

```
costFunction = @(params) dipoleCostFunction(params,
    LeadFieldMatrix, Interictal, SourceLocations, rand_index);

initialGuess = [rand*max(SourceLocations(1,:)); rand*max(
    SourceLocations(2,:)); rand*max(SourceLocations(3,:))];

initialDirection = rand(3,1);
initialDirection = initialDirection / norm(initialDirection); %
    Normalize

initialParams = [initialGuess; initialDirection];

options = optimoptions('simulannealbnd', 'Display', 'iter', '
    MaxIterations', 2000, 'FunctionTolerance', 1e-6);

[optimalParams, optimalCost] = simulannealbnd(costFunction,
    initialParams, [], [], options);

optimalLocation = optimalParams(1:3);
optimalDirection = optimalParams(4:6);

disp('Optimal Dipole Location:');
disp(optimalLocation);

disp('Optimal Dipole Direction:');
disp(optimalDirection);

% Calculate the position and direction errors
position_error_10 = norm(optimalLocation - SourceLocations(:,
    rand_index));
direction_error_10 = norm(optimalDirection - DipoleDirection);

disp("Position Error (Simulated Annealing): " + position_error_10)
    ;
disp("Direction Error (Simulated Annealing): " +
    direction_error_10);
```

```
function cost = dipoleCostFunction(params, LeadFieldMatrix,
    Interictal, SourceLocations, rand_index)
    location = params(1:3);
    direction = params(4:6);

    direction = direction / norm(direction);

    source_columns = (double(rand_index)-1)*3 + (1:3);

    lead_field = LeadFieldMatrix(:, source_columns);
    M = lead_field * direction;

    M = M(1:20 , :);


    cost = norm(M - Interictal);
end
```

the output of this code contains the rate and the values. the final error i achieved is as below:

position error is:1.753
direction error is:0.856